MOLARIUM TECHNICAL APPENDIX

Molarium operating manual

A module-by-module operating reference derived from the implementation, tests, and repository history.

Inspected 2 September 2026

Feature branch through commit ae052c4

Contents

1 Scope and status conventions

This manual describes `feature/sos1-interface-story` through commit `ae052c4` on 2026-09-02. The blind implementation/history pass covered the repository through `663e4a71578d`; subsequent factual review incorporates the route/ledger schema split and the pre-release `designRoute.*` API cleanup. The manual is derived from implementation, tests, module READMEs, and Git history. It does not describe intent that has no executable implementation.

Status terms used below:

- **Current browser surface** - reachable from the main application UI or its installed browser API. The source entry points are `index.html`, `app.js`, and `chemist-actions.mjs`; these files were byte-identical to their counterparts in `dist/` at inspection time.
- **Experimental current surface** - executable from the current browser application, but explicitly labeled experimental or subject to material constraints in the implementation or module README.
- **Repository tooling** - executable tests, command-line utilities, registries, and standalone viewers that are not general controls in the main workbench.
- **Proposed** - design text or a preregistered/development record without a corresponding general runtime path. Proposed items are not operating instructions.
- **Retired/compatibility-only** - code or assets preserved for compatibility or history but not presented as a current workbench choice.

The application is a single-page browser workbench. Its default launch loads a bundled LSD molfile; `?blank` starts with no molecule. A registered story URL instead verifies and installs an action script on a blank canvas (`app.js:13938–14008`).

2 Runtime, distribution, and network policy

Responsibility

Serve the static browser application, select connected or Local Lab behavior, and verify large runtime assets before a worker uses them.

Entry points and controls

- `bun run start` (or `bun server.js`) starts the Bun server; `bun run start:local` selects Local Lab. `--port`, `PORT`, `--local-only`, and `--test-api` alter the server mode (`server.js:1–105`; `package.json:6–16`).
- A static host uses the checked-in `runtime-config.js`, whose defaults are connected mode with the test API disabled (`runtime-config.js:1–11`).
- The project information dialog exposes the current network policy and a build-verification action (`app.js:13892–13919`).

Inputs, outputs, and state

- Input: HTTP requests for application files and, in connected mode, external structure, component-dictionary, and MSA requests.
- Output: static HTML, JavaScript, styles, model/runtime assets, and a small runtime configuration object.
- Mutation: server mode changes which features are enabled in the rendered session. It does not create a server-side project record.
- External boundary: connected mode may contact RCSB for PDB/CCD data and a configured MSA service. Local Lab permits same-origin resources only and applies restrictive response headers (`server.js:11–44`, `server.js:73–85`; `README.md:25–60`).
- Persisted records: deployment builds copy the application to `dist/`; large assets use versioned object paths and hash manifests (`DEPLOYMENT.md:1–80`).

Failure behavior and limits

- Asset fetch, size, or SHA-256 mismatch rejects the worker initialization; a rejected verification promise is cleared so a later attempt can retry (`app.js:10033–10155`).
- A normalized request path that does not resolve to a file returns an HTTP error rather than arbitrary filesystem content (`server.js:46–105`).
- Local Lab disables controls requiring external PDB, CCD, or MSA access. This is an operating mode, not an offline cache

manager.

- The repository describes Cloudflare Pages plus versioned object storage, but the checked-in source cannot establish which commit is live at an external URL. "Current" in this manual therefore means the inspected build, not a claim about an independently hosted instance.
- `--test-api` is a privileged development mode. Production intentionally exposes Chemist Actions but not the browser-test harness (`browser-test.js:122-133`; `app.js:9701-9703`).

3 Workspace shell and session state

Responsibility

Own the current molecular graph, visual view, edit transaction, undo/redo stacks, calculation results, docking state, design replay state, component visibility, and lazy worker handles.

Entry points and controls

- Main modes: View, Build, and Run (`index.html:529-531`). Structure loading remains available in the left panel.
- Viewer toolbar: Export, Clear, Share, Save, Finish chemistry, Discard, and Fullscreen (`index.html:630-637`).
- Public automation: `window.MolariumChemistActionsReady`, which resolves to the installed Chemist Actions API (`app.js:14005-14007`).

Inputs, outputs, and state mutations

- The central in-memory state object contains the molecule, camera, selections, pending chemistry, build/redo histories, calculation frames and ensembles, docking reference/results, registered design route, designer script/replay, component visibility, depiction state, and worker maps (`app.js:655-766`).
- Loading a molecule stops calculation playback, resets incompatible calculation, docking, selection, and preparation state, and chooses an initial representation appropriate to the source (`app.js:3078-3142`).
- Clear removes the molecule and all associated edit, calculation, docking, registered-route, audit, component, focus, and depiction state (`app.js:13838-13866`).

Persisted/exported records

- There is no implemented local-storage, IndexedDB, server project, or automatic session restore path.
- Share copies the current URL, or a registered story permalink. It does not serialize the current molecule or session (`app.js:13869-13875`).
- Save writes a PNG of the 3D canvas (`app.js:13877-13881`). Other exports are module-specific and listed in section 18.

Failure behavior and limits

- Reloading or clearing the page discards unsaved session state.
- Worker instances are lazy and session-scoped. Worker errors reject outstanding work, terminate the failed worker, and remove it so a later request can recreate it (`app.js:10167-10205`).
- The workbench does not provide a general project bundle containing structure, view, edit history, calculations, and docking records.

4 Structure ingestion

Responsibility

Create the current molecular graph from pasted coordinates, uploaded text, an identifier, a bundled example, or a registered story.

Entry points and controls

- Paste coordinates, Upload file, or enter an identifier (`index.html:73-115`).
- Identifier input accepts the built-in example names, a PDB identifier, or a SMILES string (`app.js:13422-13460`).
- Prepared examples are a separate selector (`index.html:178-183`).
- The launch asset is `assets/lsd-launch.mol`; failure falls back to an embedded XYZ example (`app.js:13938-13954`).

Inputs and outputs

- XYZ: optional atom-count and comment lines, then element and three coordinates. Invalid coordinate lines are ignored; at least one valid atom is required. Coordinates are centered, and bonds are inferred from covalent radii (`app.js:1204–1228`).
- MOL: the internal parser accepts V2000 atoms, bonds, and formal-charge records, then centers the result (`app.js:1231–1278`).
- PDB: only the first model is loaded. Alternate locations prefer blank or `A`, otherwise the highest occupancy. Protein templates, `SSBOND`, and `CONECT` records contribute bonds; hydrogens are assigned to the nearest same-residue partner (`app.js:1346–1458`).
- PDB identifiers are fetched from RCSB in connected mode and must be four characters with an initial digit (`app.js:13422–13431`).
- SMILES is embedded through the RDKit worker. Successful ingestion produces a molecule object and may immediately enumerate protonation states (`app.js:13434–13460`).

State mutations, boundaries, and records

- Successful input replaces the current molecule and resets dependent state via `loadMolecule()` (`app.js:3078–3142`).
- External boundary: PDB identifier retrieval uses RCSB; SMILES embedding uses the local RDKit worker and its verified assets.
- The input structure itself is not persisted automatically. Source/provenance fields remain attached to the in-memory molecule.

Failure behavior and verified limits

- Parse and fetch failures are shown as notices and leave the previous scene in place for paste/upload/identifier handlers (`app.js:13408–13479`).
- The upload control advertises `.xyz,.pdb,.mol`, but its handler calls `parseStructureInput()`, which routes only PDB versus XYZ. Uploaded MOL files therefore do not use the implemented V2000 parser (`index.html:94–99`; `app.js:1461–1465`; `app.js:13470–13479`). The bundled launch MOL is parsed through a separate explicit path.
- The MOL parser is V2000-only. The PDB loader is not a trajectory or multi-model reader.
- XYZ bond inference is proximity-based. Interactive Builder chemistry, by contrast, operates on explicit graph bonds.

5 Ligand protonation

Responsibility

Enumerate and apply ligand protonation states at an operator-selected pH.

Entry points and controls

- The Load panel exposes pH from 0 to 14, state selection, and Apply (`index.html:116–131`).
- The same functions are available through Chemist Actions under the protein/structure routes installed in the application (`app.js:8243–8711`).

Inputs, outputs, and mutations

- Input: current ligand SMILES and pH.
- RDKit matches a versioned Dimorphite-style site table, edits formal charges and hydrogens, sanitizes results, and reports an ordered state list with weights (`rdkit-worker.js:174–204`; `rdkit/README.md:23–32`).
- Apply re-embeds the chosen state, replaces the current molecular graph and coordinates, and invalidates state that depends on the prior topology.

Boundary, records, failures, and limits

- Boundary: dedicated RDKit Web Worker using pinned local runtime assets.
- Record: selected state and enumeration information exist in session state; no standalone protonation-record export is implemented.
- Errors from invalid pH, sanitization, embedding, or worker startup are surfaced and do not partially apply a state.
- Reported weights are independent Henderson-Hasselbalch estimates. They are not microscopic-pKa or coupled-site populations (`rdkit/README.md:23–32`).

6 PDB preparation and parameterization

Responsibility

Inspect a loaded PDB, apply a bounded set of repairs and protonation choices, and, if no blocker remains, create a complete numeric simulation system.

Entry points and controls

- Preparation controls select pH, histidine policy, heavy-atom repair, ligand policy, water policy, and gap policy (`index.html:133–177`).
- The right-side preparation inspector reports issues and downloads a preparation report (`index.html:686–698`).

Inputs and outputs

- Options normalize to: histidine `auto/hid/hie/hip`; repair missing supported heavy atoms; ligand `ccd/exclude`; water `crucial/retain/exclude`; and gap `cap/block` (`app.js:2884–2905`).
- Preview inventories residues, missing atoms, gaps, ligands, water, clashes, and invalid coordinates. It may retrieve CCD data, reconstruct supported missing heavy atoms, add standard hydrogens and terminal oxygen, select histidine states, and scan 24 polar-hydrogen orientations (`app.js:2946–3056`).
- Apply refuses a preview with blockers, runs the OpenMM worker's parameterization path, replaces the molecule with the prepared version, marks it `parameterized-experimental`, and adds an undo snapshot (`app.js:9965–10021`).

State mutations, boundaries, and records

- Preview updates `state.pdbPreparationPreview` without committing coordinates.
- Apply mutates the molecular graph, coordinates, preparation audit, and parameterization system.
- External boundary: CCD lookup in connected mode. Compute boundary: OpenMM worker and parameterization modules.
- Export: preparation-report JSON (`app.js:3059–3064`). The molecule also carries its preparation audit and parameterization in memory.

Failure behavior and verified limits

- Unsupported residues, unsupported missing atoms, incomplete retained ligands, blocked gaps, invalid coordinates, or severe clashes are blockers. Apply reports the blocker rather than manufacturing a production claim.
- The preview explicitly records `readyForProductionSimulation: false` (`app.js:3050`).
- Current preparation does not reconstruct missing loops, build membranes, model metal coordination, parameterize covalent ligands, or create periodic solvent boxes (`README.md:305–343`).
- Arbitrary proteins use an experimental Sage/Gasteiger route. Bundled prepared Rosemary fixtures are exact preparameterized systems and do not demonstrate general browser-native protein typing (`openff/README.md:23–58`; `README.md:305–343`).

7 Molecular viewer, components, and synchronized depiction

Responsibility

Display the same molecular graph in interactive 3D and, for a selected small-molecule component, RDKit-generated 2D; provide atom, residue, component, trajectory-frame, and replica selection.

Entry points and controls

- Pointer drag rotates; Shift-drag pans; wheel zooms. Click selects atoms. Viewer display controls select representation, color mode, labels, axes, auto-rotation, and measurement (`app.js:13728–13835`; `index.html:214–247`).
- Component controls show/hide and focus connected components (`index.html:671–683`; `app.js:2342–2412`).
- Protein controls expose pocket, contact, residue-focus, and interaction views.
- Result controls select saved frames, conformers, and STORMM replicas (`index.html:730–790`; `app.js:12398–12476`).

Inputs, outputs, and mutations

- Input: current graph, coordinates, residue annotations, component assignments, display settings, and optional frame/replica arrays.

- Output: canvas rendering, scene preview, component list, selection details, noncovalent-contact overlays, and sanitized RDKit SVG depiction.
- State mutations are view-only unless the operator invokes a Builder action. Camera, focus, visibility, selected atom(s), selected residue, frame, and replica are session state (`app.js:655–766`).

Boundary, records, failures, and limits

- The 2D depiction boundary is the RDKit worker. The 3D viewer is application canvas code.
- Export: PNG captures the 3D canvas. View settings are not included in a general session export.
- The 2D target is one component and is capped at 256 atoms (`rdkit-worker.js:207–248`). A failed depiction leaves the 3D molecule usable and shows an unavailable depiction state.
- Protein covalent atoms are protected from ordinary ligand chemistry edits. Local cleanup operates on a bounded neighborhood (`README.md:176–225`).
- Trajectory alignment is for display only; raw calculation coordinates are retained for replica/frame data and coordinate export (`app.js:11340–11662`; `README.md:176–225`).
- Pocket membership is a contact-oriented cutoff view, not a binding-site prediction. The default pocket/contact construction uses the documented 5 Å neighborhood (`README.md:176–225`).

8 Builder and chemistry transactions

Responsibility

Modify graph topology, atom properties, stereochemistry, local geometry, fragments, and selected coordinates while maintaining an undoable molecular state.

Entry points and controls

- Undo/Redo; fragment staging and templates; element replacement; geometry controls; bond order and aromaticity; formal charge; explicit hydrogen; stereochemistry; sidechain rotamers; designer moves; and optimization controls are in the Build panel (`index.html:249–408`, `index.html:701–723`).
- Geometry selection uses connected paths of two, three, or four atoms for distance, angle, or dihedral operations (`app.js:6354–6540`).

Inputs, outputs, and state mutations

- A chemistry-changing action opens or updates one `chemistryTransaction`, captures the pre-edit molecule, applies graph mutations, reconciles hydrogens, maintains stable atom identifiers, and records changed atoms (`app.js:6541–6769`).
- Finish sanitizes through RDKit, performs local coordinate cleanup, remaps reference contacts when applicable, invalidates obsolete parameterization, and pushes one undo snapshot for the transaction (`app.js:6825–6894`).
- Discard restores the pre-transaction molecule (`app.js:6897–6906`).
- Coordinate-only geometry actions move the connected side selected by the graph traversal; ring restrictions prevent ambiguous cuts.

Boundaries and records

- RDKit sanitization and eligible RDKit cleanup run in a worker. WebGPU and ANI optimization routes use their own workers.
- The in-memory build/redo stacks retain snapshots. Chemist Actions adds a bounded audit record; design replay can turn completed public actions into an exported script.

Failure behavior and verified limits

- A synchronous mutation failure restores the transaction snapshot. A Finish failure keeps the transaction pending so the operator can continue editing or discard it (`app.js:6695–6769`; `app.js:6825–6906`).
- If the final graph is valid but local coordinate polish fails, the valid chemistry remains committed and the error is stored in source metadata (`app.js:6783–6822`).
- Fused-aromatic bond editing is rejected where a local bond-order change cannot preserve the aromatic graph (`app.js:6282–6311`).
- RDKit optimization is disabled for parameterized protein complexes; pocket methods require a complex; ANI availability is separately gated (`app.js:12880–13015`).

- Optimization captures ligand valence geometry and restores the pre-optimization coordinates if the result crosses the implemented safeguard (`app.js:13139–13227`; `DESIGNER–MOVES.md:158–162`).
- The induced-fit control is experimental: it moves the ligand plus complete residues within 6 Å while fixing the outer system. It is not an affinity or ensemble calculation (`DESIGNER–MOVES.md:87–90`).

9 RDKit subsystem

Responsibility

Provide SMILES embedding, sanitization, 2D depiction, protonation, small-molecule conformer seeds, force-field energy, local geometry optimization, and short dynamics.

Entry points

- Main UI: identifier/SMILES load, 2D view, Finish chemistry, ligand optimization, RDKit Run method, and STORMM conformer seeding.
- Worker jobs: `smiles–depict`, `depict`, `protonation`, `smiles–embed`, `sanitize`, `conformers`, `energy`, `geometry`, and `dynamics` (`rdkit–worker.js:174–469`).

Inputs, outputs, state, and boundary

- Input: molecular graph or SMILES, coordinates, fixed-atom set where supported, and job settings.
- Output: sanitized graph/SMILES, SVG, embedded coordinates, conformer arrays, energies, optimized coordinates, or saved dynamics frames.
- Mutations occur only when the application applies the returned graph or coordinates.
- Boundary: dedicated Web Worker and pinned RDKit WebAssembly assets with recorded hashes (`rdkit/README.md:34–51`).

Records, failure behavior, and limits

- Calculation results store RDKit version, force field/fallback, timing, energies, and frames in session (`app.js:12620–12653`).
- Worker errors include the job identifier and reject the pending application request.
- Fixed atoms are restored exactly after the V2000 round trip used for optimization (`rdkit–worker.js:419–429`).
- Conformer count is capped at 256. Large requests can use up to four RDKit workers for seed generation (`rdkit–worker.js:250–469`; `README.md:99–101`).
- Depiction is capped at 256 atoms for the selected component.
- The RDKit dynamics path reports a 0.1 fs timestep and is a short workbench calculation, not a production trajectory generator (`rdkit–worker.js:430–469`).

10 OpenFF/Sage parameterization

Responsibility

Convert a supported molecular graph into the numeric bonded, nonbonded, charge, constraint, and optional implicit-solvent arrays consumed by calculation workers.

Entry points

- Automatic prerequisite to direct WebGPU, STORMM on the current molecule, docking, and preparation workflows.
- There is no separate general ”parameterize” button for an arbitrary ligand.

Inputs, outputs, state, and boundary

- Input: atoms, bonds, coordinates, charge, and selected simulation options.
- The parameterizer converts to a mol block, checks supported elements, loads the Sage schema, uses RDKit for SMIRKS matching and deterministic Gasteiger charges, and requires every needed parameter assignment (`openff/sage–parameterizer.js:22–339`).
- Output: a complete numeric system plus parameter labels, force-field identity, charge model, and source hash.
- Successful automatic parameterization is cached on `molecule.parameterization` (`app.js:12563–12577`). Topology-changing edits invalidate it.

- Boundary: OpenFF JavaScript modules plus RDKit worker services.

Records, failure behavior, and limits

- Parameterization metadata is shown with calculation results and retained in the current molecule.
- Unsupported elements, invalid chemistry, or an unmatched required parameter fail the request; workers do not silently omit the term.
- Charge assignment is deterministic Gasteiger, not AM1-BCC or NAGL (`openff/README.md:1–21`).
- Simulation options implement no constraints or X-H constraints, nonperiodic cutoff validation, timestep validation, and OBC2/ACE implicit solvent (`openff/simulation-options.js:14–87`; `openff/implicit-solvent.js:14–59`).

11 Direct WebGPU force-field engine

Responsibility

Evaluate the numeric Sage system directly on WebGPU for energy, geometry optimization, and dynamics.

Status and entry points

Experimental current surface. Select Direct WebGPU in Run, or use WebGPU optimization controls in Build (`index.html:412–520`; `app.js:12707–12719`).

Inputs, outputs, state, and boundary

- Input: parameterized system, coordinates, job type, step count, solvent/constraint options, saved-frame count, and optional movable atoms for geometry optimization.
- Output: positions, forces, total and component energies, convergence/timing metadata, and frames for dynamics (`webgpu-worker.js:590–699`).
- Geometry/dynamics positions replace the current coordinates; energy does not. Calculation frames and metadata populate the result card (`app.js:12552–12724`).
- Boundary: browser GPU adapter/device and verified shader/runtime assets. Buffers are destroyed after the job.

Records, failure behavior, and limits

- In-session record includes force field, charge model, solvent, constraints, cutoff, active/fixed atom counts, backend, elapsed time, energies, and frames.
- Missing WebGPU, insufficient adapter limits, invalid/nonfinite data, or worker/device errors fail the calculation and populate `lastCalculation.error` (`app.js:12727–12738`).
- The application caps direct dynamics at 5,000 steps (`app.js:12508–12530`).
- The engine uses f32 and has no periodic boundaries, PME, virtual sites, barostat, explicit-solvent setup, generic protein parameterization, or batched independent-system execution. A cutoff path exists in the numeric layer, but normal workbench workflows use the displayed nonperiodic/no-cutoff configuration (`webgpu/README.md:1–48`; `README.md:154–174`).

12 OpenMM worker

Responsibility

Run the complete numeric system with OpenMM 8.2 Reference in browser WebAssembly for internal parameterization, scoring, fixed-scaffold relaxation, validation, and compatibility workflows.

Status and entry points

Internal current subsystem. `runCalculation()` accepts an `openmm` override, but the Run method selector does not expose OpenMM; visible choices are Direct WebGPU, WebGPU ensemble, RDKit, and ANI-2x (`app.js:12508–12555`; `index.html:433–438`).

Inputs, outputs, state, and boundary

- Worker jobs include parameters, energy, geometry, dynamics, conformer benchmark/fixed-conformer work, and score (`openmm-worker.js:253–568`).

- Input is a molecule or complete numeric system. Output is parameters, energy, coordinates, frames, and platform/version metadata.
- Boundary: OpenMM 8.2 WebAssembly using the double-precision Reference platform (`openmm/README.md:1–15`).

Records, failure behavior, and limits

- Internal callers retain only the records their parent workflow stores, such as preparation audits, docking refinements, or validation results.
- Initialization, context construction, parameter, or nonfinite-result errors reject the calling workflow.
- Execution is nonperiodic. The older MolariumFF entry remains in the binary for compatibility but is not exposed in the current UI (`openmm/README.md:1–15`).

13 STORMM WebGPU ensemble

Responsibility

Run many homogeneous replicas of one topology for molecular dynamics or the anneal/minimize conformer protocol.

Status and entry points

Experimental current surface. Choose WebGPU ensemble with Dynamics or Conformer search. Controls select current/prepared system, replica count, conformer count/effort/cluster cutoff, comparison arena, and saved frames (`index.html:412–520`).

Inputs, outputs, state, and boundary

- Input: one numeric topology, initial coordinates or RDKit conformer seeds, 1-1,024 replicas, dynamics/conformer settings, constraints, and solvent.
- Conformer protocol uses deterministic seeded stages: initial relaxation, 600 K, 450 K, 300 K, and final relaxation (`openff/conformer-protocol.js:5–29`).
- Output: replica-major coordinates/frames, energies, constraint diagnostics, timing, and conformer analysis (`stormm-worker.js:187–377`).
- The selected replica or conformer updates displayed/current coordinates. Raw replica frames remain separate from display alignment (`app.js:12398–12476`).
- Boundary: WebGPU engine with deterministic integer accumulation/storage and an f32 coordinate mirror (`stormm/README.md:1–156`).

Records, failure behavior, and limits

- The result card stores replica/conformer counts, system identity, homogeneity, timing, frames, cluster analysis, and optional arena results (`app.js:12620–12724`).
- Worker validation rejects unsupported job types, replica counts, step counts, atom counts, excessive pair work, NaN values, and fixed-point overflow/clamp conditions (`stormm-worker.js:187–377`).
- Limits include one homogeneous topology, at most 512 atoms per replica, at most 1,024 replicas and 100,000 steps per request, all-pairs nonperiodic interactions, no PME/PBC, and a bounded pair-work allocation (`stormm/README.md:142–156`).
- Fixed-point spatial precision degrades when coordinates are far from the working origin; recenter structures before relying on tight positional comparisons (`stormm/README.md:142–156`).

14 ANI-2x MLIP

Responsibility

Compute ANI-2x ensemble electronic energies and analytical forces for eligible small neutral molecules, with optional geometry optimization.

Status and entry points

Experimental current surface. Choose ANI-2x with Energy or Geometry. It is also an eligible Builder optimization route when the molecule passes the gate (`app.js:12523–12530`; `app.js:12880–13015`).

Inputs, outputs, state, and boundary

- Input must be a connected, neutral, singlet molecule with explicit hydrogens, at most 96 atoms, and only H, C, N, O, S, F, or Cl (`mlip/ani2x.js:1–68`).
- Output includes total electronic energy, ensemble standard deviation, forces, optimized coordinates where requested, model hash, evaluation count, backend, and timing (`mlip-worker.js`; `app.js:12620–12719`).
- Geometry results replace current coordinates; energy results do not.
- Boundary: eight ONNX models through ONNX Runtime Web, preferring WebGPU and falling back to WASM; a CPU descriptor path is available (`mlip/README.md:1–13`).

Records, failure behavior, and limits

- Result metadata records model, level, model-source hash, backend, evaluation count, and initial/final ensemble spread.
- Eligibility failures are reported before execution. Model download/init or nonfinite results fail the request.
- No ions, radicals, disconnected systems, implicit-hydrogen inputs, unsupported elements, or molecules above 96 atoms are accepted. The result is an ANI electronic-energy model, not a Sage potential energy or binding affinity.

15 OpenFold protein prediction

Responsibility

Predict coordinates for one protein sequence using a remote MSA service and local OpenFold ONNX inference.

Status and entry points

Experimental current surface in connected mode. The Fold Protein panel accepts a sequence, MSA endpoint, and recycle settings, and exposes Run and Cancel (`index.html:188–209`). Local Lab disables the external path.

Inputs, outputs, state, and boundary

- Input: one amino-acid sequence using the standard residue alphabet or `X`, with maximum length 128. Feature buckets are 64 or 128 residues (`openfold/features.js:1–69`).
- The MSA client submits the sequence to the configured HTTP(S) endpoint, polls its ticket, validates/decompresses the returned archive, and reports rate-limit, maintenance, and timeout states (`openfold/msa-client.js`).
- The predictor loads the matching ONNX bucket, prefers WebGPU and falls back to WASM, and uses three recycles by default (`openfold/predictor.js:52–171`).
- Success loads the predicted structure as the current molecule. Export writes predicted/imported protein coordinates as PDB (`app.js:9805–9841`; `app.js:13298–13303`).

Records, failure behavior, and limits

- Session state records progress and prediction metadata. No server-side job ledger is implemented in this repository.
- Cancel aborts the active MSA/prediction controller. Fetch, archive, feature, model, and inference failures report a notice without a partial prediction (`app.js:9805–9841`).
- The sequence is transmitted to the configured MSA endpoint. First use may fetch about 540 MB of model assets (`index.html:188–209`).
- Only one entity is handled; length is capped at 128; templates and Amber relaxation are not implemented (`README.md:355–366`).

16 Calculation orchestration and results

Responsibility

Validate a Run request, prepare its engine, dispatch one worker job, apply coordinates, and present energy, progress, performance, frames, replicas, or conformer summaries.

Entry points and controls

- Job: Geometry optimization, Single-point energy, Molecular dynamics, or Conformer search.
- Method: Direct WebGPU, WebGPU ensemble, RDKit, or ANI-2x.

- Settings include steps, solvent, X-H constraints, ensemble/system controls, conformer protocol, comparison arena, and saved frames (`index.html:412–520`).

Inputs, outputs, and mutations

- Dispatch validates molecule presence, pending chemistry, concurrency, and method/job compatibility. Conformers require STORMM; STORMM otherwise supports dynamics; ANI supports energy/geometry (`app.js:12508–12530`).
- STORMM on the current molecule automatically parameterizes if needed. Conformer search first obtains RDKit seeds, then runs STORMM or the comparison arena (`app.js:12552–12619`).
- Non-energy jobs apply returned coordinates. Results populate raw/display frames, optional ensembles, `state.lastCalculation`, and the result card (`app.js:12620–12724`).

Boundary and records

- Boundary: one lazy worker per calculation family. Progress messages update the modal overlay.
- In-session records include initial/final energies, engine/model versions and hashes, force field, charge model, solvent, constraints, cutoff, convergence, platform/backend, elapsed time, timestep, frames, replicas, and conformer analysis (`app.js:12620–12653`).
- Export offers the current frame as XYZ and conformer ensembles as SDF where applicable (`app.js:9705–9768`).

Failure behavior and verified limits

- A pending chemistry transaction blocks Run. A second calculation is refused while one is active (`app.js:12515–12520`).
- Failure marks the result state `error`, stores the message in `lastCalculation`, restores the enabled Run button, and hides the progress overlay (`app.js:12727–12738`).
- The general Run overlay has no calculation-cancel control. OpenFold has its own cancellation path.
- Saved-frame choices are bounded UI samples, not a full trajectory archive (`index.html:510–517`). The current workbench has no dedicated generic trajectory-file export.
- Conformer SDF export requires V2000-sized records and otherwise reports an error (`app.js:9718–9768`).

17 Reference-guided docking

Responsibility

Carry a prepared reference pose through a local ligand edit, generate feasible candidate poses, score/rank them in a rigid local receptor site, and record the workflow in a tamper-evident labbook.

Status and entry points

Experimental current surface. In Build, capture a reference, choose Pose propagation or Expert selected core, choose cleanup/contact options and candidate count, then run constrained docking (`index.html:264–307`). The current protocol snapshots are ConstraintDock 0.4.0 and Pose Propagation 0.9.0 (`docking/README.md:56–91`).

Submodules

- `docking/protocol.mjs` - immutable protocol/settings snapshot and status.
- `docking/reference-core.mjs` - stable atom identity, reference capture, mapping, and selected-core validation. Expert cores require at least three connected, non-collinear heavy atoms (`docking/reference-core.mjs:47–80`).
- `docking/browser-adapter.mjs` - extracts ligand/receptor plans, reference contacts, and applies a compatible pose.
- `docking/feature-seeding.mjs` - deterministic edited-region torsion/feature seeds for pose propagation.
- `docking/contact-remap.mjs` - perceives reference features and proposes compatible replacements at an edit boundary.
- `docking/transformed-ring-region.mjs` and `docking/ring-conformer-generator.mjs` - identify released ring systems and enumerate bounded ring variants.
- `docking/constraints.mjs` and `docking/manual-hbond.mjs` - evaluate hard positional/core and selected hydrogen-bond constraints.
- `docking/restraint-biased-search.mjs` - first searches a restraint objective, then sends feasible poses to physical scoring.
- `docking/receptor-score.mjs` - evaluates local ligand-receptor Lennard-Jones and Coulomb cross terms.
- `docking/workflow.mjs` - aligns/snaps, relaxes, deduplicates, ranks, and returns candidate records.

- `docking/sidechain-rotamers.mjs` - enumerates a bounded set of canonical sidechain alternatives for explicit selection/application.
- `docking/labbook.mjs` - append-only event records with chained SHA-256 hashes and Markdown rendering.

Inputs, outputs, and mutations

- Capture requires a prepared complex and stores the ligand plan, stable atom identifiers, selected/reference core, receptor atoms within 8 Å, selected H bonds, and provenance (`app.js:4474-4640`).
- Run refuses pending chemistry, a concurrent docking run, or unavailable referenced contacts. It verifies receptor/contact integrity, maps surviving ligand atoms, releases transformed rings, generates seeds, freshly parameterizes the edited ligand, relaxes/scores candidates, and updates progress while periodically yielding to the UI (`app.js:4642-5428`).
- Output is a ranked list of distinct candidate poses plus completion summary and labbook. Candidates remain separate from the molecule until Apply. Apply rechecks topology and receptor integrity before replacing coordinates (`app.js:5335-5530`).

Boundaries and records

- Boundaries: RDKit seed generation, OpenFF parameterization, WebGPU/OpenMM relaxation and scoring paths, all coordinated in the browser.
- Exports: labbook JSON and human-readable Markdown/notes. The record includes input settings, hashes, mapping/contact decisions, feasibility rejections, scores, and selected pose. Coordinate-free input hashes are intentional (`docking/README.md:110-125`).

Failure behavior and verified limits

- Invalid reference core, changed receptor topology/coordinates, unmappable required atoms, unavailable contact features, parameterization error, or no feasible candidate produces an explicit failed/negative result; it does not silently apply the least-bad pose.
- Default Pose propagation maps surviving reference atoms and seeds edited regions. Expert selected-core is a narrower operator-controlled mode. Automatic MCS alignment of an independently imported analogue is not implemented (`README.md:263-267`).
- The scorer uses a rigid 8 Å receptor site, relative dielectric 4 cross terms, and relative vacuum ligand strain. It omits general receptor relaxation, solvent displacement, macrocycle/fused-ring concerted search, entropy, and binding free energy. This is local analogue pose maintenance, not global docking (`README.md:269-276`).
- The separate induced-fit Builder operation is not part of the docking ranker.

18 Chemist Actions API

Responsibility

Expose a fixed JSON-only browser API for inspection and the same structure/view/edit/docking/optimization/design operations available through application routes.

Entry point and controls

- Await `window.MolariumChemistActionsReady`, then call `api.execute({ action, args })`; `api.inspect(args)`, `api.describe()`, and `api.history()` provide the read-only shortcuts and records (`chemist-actions.mjs:149-211`; `CHEMIST-ACTIONS-API.md:1-155`).
- Action families cover session inspection, view, build, protein, selections, chemistry, history, pose/sidechains, optimization, registered design routes, and structure stories. Registered routes use `designRoute.load`, `designRoute.applyStep`, and `designRoute.inspect`; `designRoute.load` takes a `routeId` (`chemist-actions.mjs:3-105`).

Inputs, outputs, state, and boundary

- Inputs are plain JSON. Maximum nesting is 8, node count 2,048, encoded size 32,768 bytes; prototype-related keys are forbidden (`chemist-actions.mjs:107-136`).
- Installed routes call application functions, so mutations and safeguards are the same as UI operations (`app.js:8243-8711`).
- Calls are serialized through one promise queue. Recognized calls create `running`, then `completed` or `failed`, audit entries; the bounded history defaults to 500 entries (`chemist-actions.mjs:149-211`).
- Boundary: same-page JavaScript API. It does not grant arbitrary code execution, direct coordinate injection, callbacks, or a generic network function.

Persisted/exported records, failures, and limits

- Audit is session state and can be converted into a Designer action script. Completed public actions are replay candidates; failed actions remain audit evidence.
- Unknown actions fail before an audit entry because no installed route exists. A route failure records the failed call and propagates its error (`chemist-actions.mjs:149-211`).
- Inspection defaults to a ligand-oriented, coordinate-free summary; coordinates require an explicit request and are capped by the documented atom limit (`CHEMIST-ACTIONS-API.md:147-155`).
- The API is a stable command boundary, not a privileged test surface. Browser-test helpers require `--test-api` and are not part of production (`browser-test.js:122-133`).

19 Designer replay, registered routes, campaign ledgers, and standalone viewers

Responsibility

Capture completed public actions as replayable scripts; replay them deterministically through the public API; validate registered graph-edit routes; maintain separate append-only, content-addressed campaign ledgers; and render selected records in standalone viewers.

Status and entry points

- **Current browser surface:** Import script, Restart, Play/step story, and Export script controls in Build (`index.html:342-359`). Registered story URLs load a hash-verified bundled script on a blank canvas (`app.js:13957-14003`).
- **Current API surface:** registered-route actions under `designRoute.*`, plus structure-story actions (`chemist-actions.mjs:73-105`). There is no pre-release compatibility alias.
- **Standalone repository surfaces:** `design-history/viewer/` and `design-history/structure-viewer/` consume bundled records independently of the main workbench.

Inputs, outputs, and state mutations

- Replay accepts a validated script containing only public actions and binding captures. Import clears the existing molecule before installing the script (`design-history/replay.mjs`; `DESIGNER-MOVES.md:121-123`).
- Playing invokes the same public action routes as a manual/API operation and maintains replay position/status. Replay output is a separate audit record, not a rewrite of the source script.
- `actionScriptFromAudit()` includes only completed recognized public actions (`design-history/replay.mjs`).
- A registered design route uses `molarium.registered-design-route/v1`. It defines the allowed coordinate boundary, hash-pinned hit, ordered graph edits, and locked evaluation state. It is input to a prospective workflow, not a history ledger (`design-history/structures/design-route.mjs`).
- The campaign ledger stores content-addressed structures/scripts, hash-chained events, decisions, commits, branches, and finalization state. Finalized entities are immutable (`design-history/ledger.mjs`).

Boundaries and records

- Boundary: no additional scientific engine; scripts call the installed Chemist Actions boundary and therefore cross into the same workers as the invoked actions.
- Exports: action-script JSON, replay audit, and design campaign/standalone-viewer records where their specific UI or utility provides download/build output.
- Campaign actor types in the implementation are `human`, `agent`, `system`, and `import` (`design-history/ledger.mjs:28`).

Failure behavior and verified limits

- Schema, action, binding, hash, or referenced-content mismatch stops validation/replay. The route loader rejects a campaign ledger, and the ledger verifier rejects a registered route. A registered story additionally verifies its source SHA-256 before installation (`design-history/structures/design-route.test.mjs`; `app.js:13983-14002`).
- Import is destructive to the unsaved in-memory scene by design; export first if it must be retained.
- Hash chains detect record mutation but do not authenticate an author, establish scientific truth, or replace a signature (`design-history/README.md:131-144`).

- Scripts cannot bypass public action validation with direct coordinates or arbitrary JavaScript.

20 Enumeration development modules

Responsibility

Represent bounded topology-edit plans, execute them through the public API, describe graph-lineage disruption, and hold small development catalogues/panels.

Status and entry points

Repository tooling/development, not a general current workbench enumeration UI. Current implementation contains three development transformations and separates enumeration from docking (`enumerations/README.md:1–51`).

Inputs, outputs, state, and boundary

- An action plan contains supported atom/bond/charge operations and explicit Finish boundaries. Validation enforces supported heavy elements, bond orders, charges, and required transaction completion (`enumerations/action-plan.mjs:14–59`).
- Execution resolves step aliases, calls only the public Chemist Actions API, and returns an execution audit plus final inspection (`enumerations/action-plan.mjs:62–105`).
- Edit-difficulty output describes graph-lineage disruption; it is not an affinity, synthesis, or MCS score (`enumerations/edit-difficulty.mjs:57–130`).
- Pose assessment produces conservative flags for downstream review (`enumerations/pose-assessment.mjs:1–26`).

Records, failures, and limits

- Development catalogues and preregistered high-disruption panels are files in `enumerations/`; Node validation includes a 7KPA-specific catalogue path (`enumerations/catalogue.mjs`).
- Invalid plans fail before execution. Runtime chemistry failures are retained in the execution audit through the public API.
- Combinatorial enumeration, general library management, global pose transfer, and automated campaign selection remain development/proposed work rather than normal UI functions (`enumerations/README.md:38–51`).

21 Validation, tests, and maintainer controls

Responsibility

Maintain versioned scientific reference records, run deterministic unit/integration/browser checks, and display the validation ledger in the application information dialog.

Entry points

- The Validation project panel lazy-loads `validation/dashboard.mjs` (`app.js:13895–13911`).
- `package.json` defines unit, scientific, browser, build, and specialized validation scripts (`package.json:6–93`).
- CI builds the production distribution and runs scientific and browser suites (`.github/workflows/ci.yml:18–76`).

Inputs, outputs, state, and boundary

- Registry v0.2 separates targets, reference systems, cases, poses, assertions, outcomes, and hashes. Versions are append-only; failed cases remain records (`validation/README.md:1–26`).
- Dashboard input is the checked-in registry; output is a read-only rendered summary. Failure renders ”Validation ledger unavailable” rather than hiding the panel (`app.js:13895–13902`).
- Browser tests may use the separately enabled test API. Maintainer scripts use Node/Bun and, for browser suites, a browser process.

Records, failures, and limits

- At inspection, registry validation reported 25 cases, 18 reference systems, 15 targets, and 5 crystal-scored cases; these are repository records, not an external certification (`README.md:368–387`).
- Registry hashes establish internal integrity. They do not make all cases independent or establish broad accuracy outside the represented systems.
- The main workbench dashboard is informational; it does not launch validation jobs or alter the registry.

22 Export and persistence matrix

| Record | How produced | Format | Contains | Does not contain |

|—|—|—|—|—|

| Current coordinates | Export | XYZ (`app.js:9705–9708`) | Current displayed molecule coordinates | Full topology metadata, complete session, calculation history |

| Conformer ensemble | Result export | SDF/V2000 (`app.js:9718–9768`) | Exportable conformer coordinates and molecular graph | Records too large for V2000; general dynamics trajectory |

| Protein structure | Protein export | PDB (`app.js:13298–13303`) | Imported/predicted protein coordinates and supported annotations | Preparation project bundle or prediction inputs/worker cache |

| Preparation report | Preparation inspector | JSON (`app.js:3059–3064`) | Options, issues, audit/ready status | A production-readiness certification |

| Docking labbook | Docking results | JSON and Markdown (`app.js:5522–5530`) | Settings, mapping, candidates, failures, hashes, selection | A cryptographic author identity; binding free energy |

| Designer script/replay | Designer controls | JSON | Public actions, bindings, replay/audit data | Arbitrary code, direct coordinate injection, complete UI state |

| Canvas image | Save | PNG (`app.js:13877–13881`) | Current 3D rendering | Molecularly reusable coordinates/topology |

| Share link | Share | Clipboard URL (`app.js:13869–13875`) | Current URL or registered story permalink | Unsaved molecule or session serialization |

| Chemist action history | API inspect/export path | In-memory JSON/action script | Bounded recognized call audit | Persistence across reload unless explicitly exported |

There is no implemented autosave, database, browser project store, generic trajectory-file export, or one-click complete-session archive.

23 End-to-end operating sequence

A. Start from a blank canvas

1. Open the application with `?blank`. Confirm the viewer is empty. Without `?blank`, the bundled launch molecule loads automatically (`app.js:13938–13974`).
2. If network provenance matters, open the network-policy panel. In Local Lab, expect PDB/CCD/MSA controls to be unavailable.

B. Load a structure

1. Choose one input path:

- Paste XYZ or PDB text.
- Upload an XYZ or PDB file.
- Enter SMILES or a PDB identifier in connected mode.
- Select a prepared example.

1. Check atom count, formula, charge, components, and the 3D graph. For a ligand, check the synchronized 2D depiction. Do not rely on ordinary `.mol` upload; the current upload router does not invoke the MOL parser.
2. For a ligand, inspect pH and protonation candidates; Apply the intended state before topology editing or parameterization.
3. For a PDB, open preparation details, set policies, Preview, inspect every blocker/warning, and download the report. Apply/parameterize only if the bounded workflow meets the intended use. The report remains explicitly non-production-ready.

C. Inspect and edit

1. Use Components and residue/pocket focus to isolate the intended working region. Set representation and contacts as needed; these are view state.
2. Enter Build. Select atoms in 2D or 3D and apply element, bond, charge, hydrogen, stereochemistry, fragment, or geometry operations.

3. For graph chemistry, keep all related edits in the pending transaction. Choose Finish chemistry to sanitize/reconcile/polish and create one undo point. If validation fails, correct the pending edit or Discard to restore its starting graph.
4. Use Undo/Redo for committed changes. If a reference pose is required, capture it before the edit that creates the analogue. Use explicit stable/core/contact selections as needed.

D. Calculate or maintain a pose

1. Ensure no chemistry transaction is pending.
2. For ordinary calculation, enter Run and choose a compatible pair:

- RDKit: energy, geometry, or short dynamics.
- Direct WebGPU: energy, geometry, or bounded dynamics.
- WebGPU ensemble: dynamics or conformer search.
- ANI-2x: energy or geometry for an eligible molecule.

1. Set steps, solvent, constraints, replicas/conformer protocol, and saved frames. Run. On success inspect engine/model identity, energy, convergence, timing, frames, and replica/conformer summaries. On failure, read the notice; the Run button is restored.
2. For reference-guided analogue docking, use Build instead: verify the captured reference, choose Pose propagation or an Expert core, review contact mappings, and Run constrained docking. Inspect distinct candidates and the labbook. Apply only the chosen candidate; until then the current molecule is unchanged.

E. Export and replay

1. Export the chemically reusable record needed by the next step: current XYZ, conformer SDF, protein PDB, preparation report, or docking JSON/Markdown. Save PNG only as a visual record.
2. For reproducible UI/API operations, export a Designer action script or campaign/replay record. An action script contains public operations, not a complete binary session snapshot.
3. To replay, first export any current unsaved work. Importing a script clears the scene. Restart the script, then Play/step it. Verify the replay audit and resulting structure; export the new records explicitly.
4. Before closing or reloading, export every needed record. Session state is otherwise lost.

24 Failure and recovery rules

- **Input error:** previous scene normally remains. Correct the text, file type, identifier, or network condition and retry.
- **Pending chemistry error:** edit validation keeps the transaction open. Correct it or Discard; do not Run calculations or docking while it is pending.
- **Calculation error:** the current graph remains, except coordinates already applied only after a successful worker return. Read `lastCalculation.error`, correct eligibility/options/assets, and retry (`app.js:12552–12738`).
- **Worker crash/message error:** all pending requests for that worker reject; the instance is terminated and removed. A later request creates a fresh worker (`app.js:10167–10205`).
- **Asset hash error:** the job stops. Restore the expected versioned asset or manifest rather than bypassing verification (`app.js:10033–10155`).
- **Docking infeasibility:** no pose is applied. Repair the reference/core/contact mapping or chemistry, then rerun. Retain the negative labbook record when provenance matters.
- **Replay/hash error:** stop. Confirm schema, registered source path, and hash. Do not treat a partially replayed session as equivalent to a completed audit.
- **Preparation blockers:** do not force Apply. Change preparation policy or prepare the structure with a system that supports the missing chemistry.
- **Unsaved state loss:** no automatic recovery path exists after Clear, page reload, or script import.

25 Current, experimental, proposed, and retired boundaries

Current browser behavior

- Structure load from XYZ/PDB/SMILES/PDB identifier and prepared examples.

- Synchronized 3D/2D inspection, component and residue focus, explicit graph editing, chemistry transactions, and undo/redo.
- RDKit calculation; export of current coordinates, conformers, proteins, reports, docking labbooks, action scripts, and images.
- Chemist Actions, Designer replay, registered hash-verified stories, and the read-only validation dashboard.

Experimental behavior that is nevertheless executable

- Browser PDB preparation/general Sage-Gasteiger protein parameterization.
- Direct WebGPU force-field calculation.
- STORMM homogeneous WebGPU ensemble dynamics/conformer search and Conformer Arena.
- ANI-2x browser inference.
- OpenFold prediction with an external MSA boundary.
- Reference-guided docking, contact remapping, ring release, bounded sidechain alternatives, and induced-fit local optimization.
- Design-campaign/history records and standalone viewers, within their stated integrity limits.

Repository tooling/development only

- Enumeration action-plan/catalogue modules and preregistered high-disruption panels.
- Browser test API, registry validators, scientific benchmarks, pose-review/build scripts, and CI commands.
- OpenMM as an internal/reference worker rather than a selectable current Run method.

Proposed or unavailable

- General remote CUDA execution is not an installed runtime path.
- GFN/xTB is unavailable in the current workbench (`README.md:389–408`).
- General combinatorial enumeration and automated campaign selection are not current UI workflows.
- Missing-loop construction, membrane/metal/covalent-ligand preparation, periodic explicit solvent, PME, binding free energy, global docking, and production-scale trajectories are not implemented as end-user workflows.

Retired or compatibility-only

- The OpenMM binary retains the older MolariumFF entry only for compatibility; it is not exposed (`openmm/README.md:13–15`).
- `free-local` cleanup remains an explicit older option; current reference-guided operation defaults to preserving the reference relationship (`index.html:287–294`).
- Git history shows the docking name change to ConstraintDock (`f9c31db`), the addition of synchronized 2D editing (`85d0b4a`), design history (`0639144`), Chemist Actions/replay (`0a7eeb3`), STORMM display-only alignment (`5621b8e`), manual hydrogen bonds (`c879077`), and the validation dashboard (`f8acdeb`). Earlier draft benchmark manifests and narrative prototypes are not current runtime contracts.

26 Verification performed for this manual

The following current repository checks passed during inspection:

- `node chemist-actions.test.mjs`
- `node docking/test.mjs`
- `node docking/manual-hbond.test.mjs`
- `node docking/restraint-biased-search.test.mjs`
- `node docking/ring-conformer-generator.test.mjs`
- `node docking/sidechain-rotamers.test.mjs`
- `node enumerations/enumerations.test.mjs`
- `node design-history/design-history.test.mjs`
- `node design-history/replay-audit.test.mjs`
- `node design-history/interface-story.test.mjs`
- `node design-history/viewer/model.test.mjs`
- `node design-history/structure-viewer/timeline.test.mjs`

- `node design-history/structures/design-route.test.mjs`
- `node validation/validate-registry.mjs`
- `node validation/dashboard.test.mjs`

These checks establish that the inspected contracts pass their repository tests. They do not expand the operating limits listed above.